agentins_tb_4k_w_consts评测标准文档背景：

背景：

每个prompt（instructions）有4条数据，分别来自不同的框架 claudecode、sweTrae、codeact、terminus2

	task_id	instance_id	instructions	framework
1	abhay-joshi-09-30-2025	abhay-joshi-09-30-2025	Build an executable Ba...	ClaudeCode
2	abhay-joshi-09-30-2025	abhay-joshi-09-30-2025	Build an executable Ba...	CodeActAgent
3	abhay-joshi-09-30-2025	abhay-joshi-09-30-2025	Build an executable Ba...	SWETraeAgent
4	abhay-joshi-09-30-2025	abhay-joshi-09-30-2025	Build an executable Ba...	Terminus2Agent

约束内容是根据instructions、constraints_category（约束类型）、constraints_task_dependent（任务相关性）、constraints_framework_dependent（框架相关性）生成的。

评测标准：

1. **第一步评分**，我们需要结合instructions、constraints_category（约束类型）、constraints_task_dependent（任务相关性）、constraints_framework_dependent（框架相关性）这些字段对约束内容打分。检查项参考 Coding Agent约束质量质检标准

分数	判定标准	例子
1分	1. 约束表述模糊、矛盾、存在 **致命错误**，无法指导代码实现，直接导致任务无法完成； 2. 约束存在 **关键错误**，违背核心需求，即使执行也无法达成目标。	
2分	1. 仅包含 **无影响的微小笔误 / 格式错误**（如错别字、标点错误、非核心参数的数值笔误），不改变约束的核心要求，执行后不影响整体结果	编写完 check.sh 后，立即通过 Bash 运行 `bash -n check.sh`，在认为实现完成之前捕获语法错误。

	2. 约束表述精准，无任何错误，且属于 **简单约束范畴** （只要求执行一个明确、直接的操作，或做一次无需额外推理的直接判断即可满足的约束；不涉及多个关联步骤，也不涉及规则聚合、去重、优先级、条件分支或例外处理。）	
3分	1. 约束表述精准，无任何错误，且属于 **复杂约束范畴** （需要多个关联步骤才能完成，或虽然表面上只有一个步骤，但需要结合多条规则进行推理、聚合、去重、优先级判断、条件分支或例外处理后才能正确满足的约束。）	必须使用 Bash 工具对比汇编输出与 C 输出（如 diff 或逐行比较），并将比较结果中每个字段的 match 值写入 verification.json，all_match 必须反映所有字段均匹配的真实状态。

2. **第二步**，每条prompt数据，最好是四个框架的约束一起看，因为可能会存在重复冗余的情况，这是一个需要仔细判断点，如果存在重复冗余的约束，四个框架打分的优先级是 claudecode > sweTrae > codeact > terminus2

   4 个框架约束重复

   claudecode=3 分，sweTrae=2 分，codeact=1 分，terminus2=1 分**（该条约束第一步评分3分时）**

   claudecode=2 分，sweTrae=1 分，codeact=1 分，terminus2=1 分**（该条约束第一步评分2分时）**

   claudecode=1 分，sweTrae=1 分，codeact=1 分，terminus2=1 分**（该条约束第一步评分1分时）**

   3 个框架约束重复

   组合 1：claudecode + sweTrae + codeact 重复

   claudecode=3 分，sweTrae=2 分，codeact=1 分**（该条约束第一步评分3分时）**

   claudecode=2 分，sweTrae=1 分，codeact=1 分**（该条约束第一步评分2分时）**

   claudecode=1 分，sweTrae=1 分，codeact=1 分**（该条约束第一步评分1分时）**

   组合 2：claudecode + sweTrae + terminus2 重复

   claudecode=3 分，sweTrae=2 分，terminus2=1 分**（该条约束第一步评分3分时）**

   claudecode=2 分，sweTrae=1 分，terminus2=1 分**（该条约束第一步评分2分时）**

   claudecode=1 分，sweTrae=1 分，terminus2=1 分**（该条约束第一步评分1分时）**

   组合 3：claudecode + codeact + terminus2 重复，

   claudecode=3 分，codeact=1 分，terminus2=1 分，**（该条约束第一步评分3分时）**

   claudecode=2 分，codeact=1 分，terminus2=1 分，**（该条约束第一步评分2分时）**

claudecode=1 分，codeact=1 分，terminus2=1 分，**（该条约束第一步评分1分时）**

组合 4：sweTrae + codeact + terminus2 重复

sweTrae=3 分，codeact=2 分，terminus2=1 分**（该条约束第一步评分3分时）**

sweTrae=2 分，codeact=1 分，terminus2=1 分**（该条约束第一步评分2分时）**

sweTrae=1 分，codeact=1 分，terminus2=1 分**（该条约束第一步评分1分时）**

2 个框架约束重复

组合 1：claudecode + sweTrae 重复

claudecode=3 分，sweTrae=2 分**（该条约束第一步评分3分时）**

claudecode=2 分，sweTrae=1 分**（该条约束第一步评分2分时）**

claudecode=1 分，sweTrae=1 分**（该条约束第一步评分1分时）**

组合 2：claudecode + codeact 重复

claudecode=3 分，codeact=2 分**（该条约束第一步评分3分时）**

claudecode=2 分，codeact=1 分**（该条约束第一步评分2分时）**

claudecode=1 分，codeact=1 分**（该条约束第一步评分1分时）**

组合 3：claudecode + terminus2 重复

claudecode=3 分，terminus2=2 分**（该条约束第一步评分3分时）**

claudecode=2 分，terminus2=1 分**（该条约束第一步评分2分时）**

claudecode=1 分，terminus2=1 分**（该条约束第一步评分1分时）**

组合 4：sweTrae + codeact 重复

sweTrae=3 分，codeact=2 分**（该条约束第一步评分3分时）**

sweTrae=2 分，codeact=1 分**（该条约束第一步评分2分时）**

sweTrae=1 分，codeact=1 分**（该条约束第一步评分1分时）**

组合 5：sweTrae + terminus2 重复

sweTrae=3 分，terminus2=2 分**（该条约束第一步评分3分时）**

sweTrae=2 分，terminus2=1 分**（该条约束第一步评分2分时）**

sweTrae=1 分，terminus2=1 分**（该条约束第一步评分1分时）**

组合 6：codeact + terminus2 重复

codeact=3 分，terminus2=2 分**（该条约束第一步评分3分时）**

codeact=2 分，terminus2=1 分**（该条约束第一步评分2分时）**

codeact=1 分，terminus2=1 分**（该条约束第一步评分1分时）**

3. 对于非3分的约束，在 **非满分备注** 字段适当写一下备注。

   可以参考：

> 🌟 约束1：与 CodeActAgent 约束2、SWETraeAgent 约束1、Terminus2Agent 约束1存在跨框架重复，核心都是“修改前先盘点项目中的 .tf 文件，避免遗漏 Terraform 配置”；4个框架重复，ClaudeCode 优先级最高，但该约束属于简单前置检查，给2分。
>
> 约束2：与 SWETraeAgent 约束4存在部分重复，核心都是“修复跨文件资源引用时先全局搜索相关引用/标签，避免改坏其他引用关系”；ClaudeCode 优先级较高，但该约束属于简单引用核查，给2分。
>
> 约束4：与 CodeActAgent 约束5、Terminus2Agent 约束3存在跨框架重复，核心都是“不得在 .tf 或变量默认值中硬编码 AWS 凭证/敏感值”；3个框架重复，ClaudeCode 优先级最高，但该约束属于简单安全约束，给2分。
>
> 约束5：要求 terraform validate 出现资源引用错误时，先全局 grep 该标识符，再判断是否真的缺失；必要性和结果可验证性成立，但属于简单排错流程，给2分。

4. 四个框架对应可以使用的工具供参考

代码块

```
ClaudeCode = """### ClaudeCode 内置工具参考
| 工具名 | 功能 | constraint 构造思路 |
|--------|------|-------------------|
| Read | 读取文件内容 | 改文件前先读？读完整文件还是部分？ |
| Write | 创建或覆盖写入文件 | 什么时候该用 Write 什么时候不该？ |
| Edit | 对已有文件做局部编辑 | 是否应该优先用 Edit 而非 Write？ |
| Bash | 执行shell命令 | 改完代码后运行什么验证？哪些命令不该执行？ |
| Glob | 按模式搜索文件名 | 操作文件前是否应该先确认文件存在？ |
| Grep | 在文件内容中搜索文本 | 修改某个函数/变量前是否应该先搜索所有引用？ |
| TodoWrite | 管理TODO列表 | 任务是否复杂到值得先列计划？ |"""
SWETraeAgent = """### SWETraeAgent 内置工具参考
| 工具名 | 功能 | 常见 constraint 示例 |
|--------|------|---------------------|
| `search_by_regex` | 按正则在目录中搜索文本 | "修改函数前先 search_by_regex 查找所有调用方" |
| `view_files` | 批量查看多个文件内容 | "先用 search 定位，再用 view_files 查看上下文" |
| `list_dir` | 列出目录结构（最深 5 层） | "用 list_dir 了解项目结构，不要猜测路径" |
| `write_to_file` | 创建新文件或覆盖已有文件 | "写前必须确认文件是否存在" |
| `update_file` | 对已有文件做局部编辑 | "用 update_file 做精准替换，不要用 write_to_file 重写整个文件" |
| `rename_file` | 重命名或移动文件 | "移动前确认目标路径不存在同名文件" |
```

```
| `delete_file` | 批量删除文件 | "删除时必须用 delete_file，不要用 shell 的 rm" |
| `run_command` | 执行 shell 命令 | "每次修改后用 run_command 运行测试验证" |
| `check_command_status` | 查询异步命令的运行状态和输出 | "同一 command_id 不得调用超过 3 次" |
| `stop_command` | 终止正在运行的命令 | "重启服务前先 stop_command 终止旧进程" |
| `todo_write` | 管理 TODO 列表 | "多步骤任务第一步必须用 todo_write 列出计划" |
| `finish` | 标记任务完成 | "所有 todo 状态为 completed 后才调用 finish" |
> `todo_write` 是无状态工具，每次传完整列表覆盖，同一时刻仅一个任务可为 `in_progress`。"""
CodeActAgent = """### CodeActAgent 内置工具参考
| 工具名 | 功能 | 常见 constraint 示例 |
|--------|------|---------------------|
| `execute_bash` | 在终端执行 bash 命令（支持 `C-c` 中断、空命令查看追加日志） | "每次代码修改后用 execute_bash 运行测试" |
| `str_replace_editor` | 查看/创建/编辑文件（view、create、str_replace、insert、undo_edit） | "不能对已存在的文件用 create" |
| `finish` | 标记任务完成，支持多种提交变体（默认/整数/文件列表/行号/IO预测） | "确认所有测试通过后才调用 finish" |"""
Terminus2Agent = """### Terminus2Agent 内置工具参考
Terminus2 不使用独立的工具调用，而是通过 **tmux 终端交互**模式运行——LLM 输出 JSON 格式的命令序列，直接发送到 tmux 会话执行。
| 操作 | 功能 | 常见 constraint 示例 |
|------|------|---------------------|
| `analysis` | 分析当前终端状态和问题 | "先分析终端输出中的错误信息，再决定下一步" |
| `plan` | 描述下一步执行计划 | "plan 中说明要执行什么命令以及预期结果" |
| `commands.keystrokes` | 向 tmux 终端发送按键/命令 | "多条命令时每条必须以 `\n` 结尾" |
| `commands.duration` | 命令执行后等待秒数 | "编译等耗时命令设置足够的 duration，避免超时" |
| `task_complete` | 标记任务完成 | "确认测试全部通过后才设 task_complete: true" |
> 交互范式：LLM 看到终端屏幕状态 → 输出 JSON（analysis + plan + commands） → 逐条发送到 tmux → 获取增量输出 → 循环。没有独立的文件编辑/搜索工具，所有操作都通过 shell 命令完成，使用 JSON 解析模式。"""
```